Lab 9: Structures and Unions (with Pointers)

Computer Programming (010153002) | Semester 1/2026

Duration: 2 hours (in-lab) | **Topic:** User-defined types, `->` member access, passing structs by pointer

Objectives

- Define a `struct` that groups related fields under one name.
- Access members with `.` on a value and `->` on a pointer.
- Pass structs to functions *by pointer* to avoid copying and to allow modification.
- Recognize when a `union` or `enum` is the right tool.

Quick Reference

Struct definition and member access.

```c
typedef struct {
    int x, y;
} Point;

Point  p  = {3, 4};
Point *pp = &p;
p.x  = 10;            /* through a value:   "." */
pp->x = 20;           /* through a pointer: "->"  -- equivalent to (*pp).x */
```

Pass by pointer (no copy, caller can be modified).

```c
void move(Point *p, int dx, int dy) {
    p->x += dx;
    p->y += dy;
}
int main(void) {
    Point p = {0, 0};
    move(&p, 3, 4);   /* p is now {3, 4} */
}
```

Why use `->`? `p->x` reads as "the `x` of the struct that `p` points to". It is purely sugar for `(*p).x` – the parentheses are needed because `.` binds tighter than `*`.

Array of structs + pointer walk.

```c
Student class[N];
for (Student *s = class; s < class + N; s++) printf("%s\n", s->name);
```

Union. All members share the same memory; only one is valid at a time.
Enum. Names a small set of named integer constants.

Quick Experiments (Try It)

Try It 1: Pass by value vs. pass by pointer: which one sticks?

Run and compare the two calls:

```c
#include <stdio.h>
typedef struct { int x, y; } Point;
```

```c
void move_by_value  (Point p,  int dx, int dy) { p.x += dx;  p.y += dy;  }
void move_by_pointer(Point *p, int dx, int dy) { p->x += dx; p->y += dy; }

int main(void) {
    Point a = {1, 2};
    move_by_value(a, 10, 10);
    printf("after by-value:   (%d, %d)\n", a.x, a.y);   /* unchanged? */

    move_by_pointer(&a, 10, 10);
    printf("after by-pointer: (%d, %d)\n", a.x, a.y);   /* changed? */
    return 0;
}
```

1. Which call actually moved the point?
2. Add `printf("struct size: %lu\n", (unsigned long)sizeof(Point));`. Now change the struct to `int data[100];`. How large is the struct now? What would the by-value copy cost each call?

Try It 2: . vs. -> – what does the compiler say?

Deliberately introduce the wrong operator and read the error:

```c
#include <stdio.h>
typedef struct { int val; } Node;
int main(void) {
    Node  n  = {42};
    Node *np = &n;

    printf("%d\n", n.val);    /* correct: value, use . */
    printf("%d\n", np->val);  /* correct: pointer, use -> */

    /* now swap them intentionally and see the errors: */
    /* printf("%d\n", np.val);  */  /* uncomment one at a time */
    /* printf("%d\n", n->val);  */
    return 0;
}
```

Uncomment each bad line one at a time. What compiler error do you get for each? Relate the error message to the rule: `.` for values, `->` for pointers.

Try It 3: Union: one block of memory, two interpretations

Run and explain why the numbers look strange:

```c
#include <stdio.h>
union U { int i; float f; };
int main(void) {
    union U u;
    u.f = 1.0f;
    printf("write float 1.0, read as int:   %d\n", u.i);
    u.i = 0;
    printf("write int 0,   read as float:   %f\n", u.f);
    printf("size of union: %lu\n", (unsigned long)sizeof(u));
    return 0;
}
```

1. What is the size of the union? Is it the sum of both members or the largest?

2. Why does reading `u.i` after writing `u.f` give a strange integer?
3. When is a union the right data structure to use? (Hint: Lab 9 Exercise 4 uses a *tagged* union – explain why the tag is essential.)

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: Point Distance + `move` via pointer *(25 min)*

Define `typedef struct { double x, y; } Point;`. Write
- `double distance(Point a, Point b)` – by value, since it only reads.
- `void move(Point *p, double dx, double dy)` – by pointer, since it writes.

Read two points, print their distance, then move the first one and print again.

In-Lab Exercise 2: Student Record – read into pointer *(30 min)*

Define a `Student` struct with `id` (int), `name` (char[20]), `gpa` (double). Write `void read_student(Student *s)` that fills the struct via `s->id`, `s->name`, `s->gpa`. Read N students ($N \leq 30$) into an array and print the one with the highest GPA.

In-Lab Exercise 3: Sort by GPA via Pointer Swaps *(35 min)*

Extend the previous exercise: sort the array in *descending* order of GPA. Use selection sort and a helper `void swap(Student *a, Student *b)` that swaps two struct values through their pointers. Print the sorted list, one student per line.

In-Lab Exercise 4: Tagged Union: Shape Area *(30 min)*

Define an enum `Kind { CIRCLE, RECT }` and a struct `Shape` containing a `Kind kind` and a `union` with the parameters of each shape (`double r` for the circle, `double w, h` for the rectangle). Write `double area(const Shape *s)` that uses a `switch` on `s->kind`.

AI Collaboration

Try these prompts with an AI tool:
- "Explain the difference between accessing a struct member with `.` versus `->` – when do I use each?"
- "I have a `Student` struct with a `char name[20]` field. Why can't I assign `s.name = "Alice"` and what should I use instead?"
- "Show me how to sort an array of structs by a numeric field using selection sort, passing struct pointers to a swap function."

Critical check – verify, don't just accept: Ask the AI to write a `typedef struct` with at least three fields and a function that fills it via a pointer parameter. Check the AI's code for: (a) correct use of `->` vs. `.`, (b) `strcpy` rather than `=` for string fields, (c) `&` at every call site. Compile with `-Wall` and fix any warning the AI's code triggers.

Know the limit: AI tools frequently use direct string assignment (`s.name = "Alice"`) in generated struct code – this is invalid for `char` array fields; the correct call is `strcpy`, and the AI may not mention buffer size safety without prompting.

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: Value copy vs. pointer

What does this print? Explain why `p` did not change.

```c
#include <stdio.h>
typedef struct { int a, b; } P;
void set_by_value(P x)   { x.a = 99; }
void set_by_ptr  (P *x)  { x->a = 99; }
int main(void) {
    P p = {1, 2};
    set_by_value(p);
    printf("%d %d\n", p.a, p.b);
    set_by_ptr(&p);
    printf("%d %d\n", p.a, p.b);
    return 0;
}
```

Trace & Predict 2: -> versus .

Predict the output:

```c
#include <stdio.h>
typedef struct { int x, y; } V;
int main(void) {
    V  v  = {7, 8};
    V *pv = &v;
    pv->x = 100;
    (*pv).y = 200;
    printf("%d %d\n", v.x, v.y);
    return 0;
}
```

Find the Bug 1: Wrong member access and missing braces

List the bugs and rewrite the corrected program:

```c
#include <stdio.h>
typedef struct { int w, h; } Rect;
int area(Rect *r) { return r.w * r.h; }
int main(void) {
    Rect r = 4, 5;
    printf("%d\n", area(r));
    return 0;
}
```

Write Code on Paper 1: Complex addition via pointer

Define `typedef struct { double re, im; } Complex;` and write `void add(const Complex *a, const Complex *b, Complex *out)` that stores $a + b$ in `*out`. Write a com-

plete program that reads two complex numbers, adds them, and prints the result as `a+bi`.

Write Code on Paper 2: Find oldest student

Define a `Student` struct with `name` and `age`. Write `Student *oldest(Student *arr, int n)` returning a pointer to the oldest student in the array. Demonstrate from `main` with three students.

Draw on Paper 1: Struct memory layout

Draw the memory layout of `Student` below as a row of labelled boxes, sized proportionally (`char[20]` = 20 bytes, `int` = 4 bytes, `double` = 8 bytes). Mark any **padding** bytes with **X**. Write the *byte offset* of each field from the start of the struct.

```
typedef struct {
    char   name[20];
    int    id;
    double gpa;
} Student;
```

Then draw a second diagram showing an *array* `Student arr[3]` in memory: three consecutive `Student` blocks, each labelled `arr[0]`, `arr[1]`, `arr[2]`, with an arrow from a pointer `Student *p = arr` to `arr[0]`.